

Auth & Session Security Analyzer

Rapport d'audit de securite Android

Application analysee : dvba
Type d'authentification : JWT
Vulnerabilites detectees : 33
Date d'analyse : 17/05/2026 11:55

Score global de securite : 0 / 10 (CRITIQUE)

1. Resume Executif

Ce resume presente les principales vulnerabilites identifiees lors de l'audit, evalue le niveau de risque global et formule les recommandations prioritaires.

L'application Android auditée est une application bancaire (DamnVulnerableBank) utilisant des tokens JWT pour la gestion de l'authentification des utilisateurs. L'analyse révèle deux vulnérabilités de niveau critique particulièrement préoccupantes : la présence de secrets et de mots de passe codés en dur directement dans le code source (fichiers b.java et n1.java), combinée au stockage des tokens JWT d'accès en clair dans les SharedPreferences Android sans aucun chiffrement, exposant l'ensemble des sessions utilisateurs à une extraction triviale. Le niveau de risque global de l'application est évalué comme critique, avec des scores nuls sur les domaines d'authentification (AUTH : 0/10) et de stockage (STORAGE : 0/10), ce qui témoigne d'une absence quasi-totale de contrôles de sécurité sur ces surfaces d'attaque fondamentales. L'impact potentiel pour les utilisateurs est majeur : un attaquant disposant d'un accès physique à l'appareil, ou exploitant une application malveillante installée conjointement, peut extraire les tokens JWT, usurper l'identité des victimes et réaliser des opérations bancaires frauduleuses sans connaissance du mot de passe. Par ailleurs, l'écriture systématique d'informations sensibles dans les logs Android et la détection d'une URL HTTP en clair aggravent la surface d'attaque en facilitant l'interception passive des données d'authentification. Les trois recommandations prioritaires sont les suivantes : premièrement, migrer immédiatement le stockage des tokens JWT vers le Android Keystore avec chiffrement AES-256 ; deuxièmement, supprimer tous les secrets codés en dur et les externaliser vers un système de gestion de secrets sécurisé côté serveur ; troisièmement, mettre en place une politique d'expiration et de révocation des tokens JWT côté backend, assortie d'une rotation systématique des refresh tokens.

2. Analyse Statique - Vulnerabilites detectees

L'analyse statique consiste en la decompilation de l'APK avec JADX et l'examen des sources Java/Kotlin par patterns regex. Les resultats sont classes par severite : CRITIQUE (-3 pts), ELEVE (-2 pts), MOYEN (-1 pt).

Fichiers analyses : 1165 | Vulnerabilites : 32 | Score : 0/10

Severite	Type	Fichier	Detail
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Parent handler not f...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager n...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager ...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Got null intent from ...
MOYEN	Communication en clair (HTTP)	a.java	http://s
ELEVE	Identifiants dans les logs	DeviceCredentialHandler...	Log.e("DeviceCredentialHandler", "onCreate: Executor and/or call...
ELEVE	Identifiants dans les logs	DeviceCredentialHandler...	Log.e("DeviceCredentialHandler", "onActivityResult: Bridge or call...
CRITIQUE	Secret code en dur	b.java	password: "); sb.append(this.f864a.isPassword()); sb.a...
ELEVE	Token dans SharedPreferences	h.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	l.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	t.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	w.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to read GetTokenResponse ...
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to convert GetTokenRespon...

CRITIQUE	Secret code en dur	n1.java	secret="); sb.append(this.i); sb.append("
ELEVE	Identifiants dans les logs	i.java	Log.e(aVar.f1410a, aVar.a("Error parsing token claims", new Obj...
ELEVE	Token dans SharedPreferences	j.java	getSharedPreferences("com.google.firebase.auth
ELEVE	Identifiants dans les logs	k.java	Log.e(aVar2.f1410a, aVar2.a("Unable to decode token", new Obj...
ELEVE	Token dans SharedPreferences	n.java	SharedPreferences.Editor editorEdit = context.getSharedPreferences...
ELEVE	Token dans SharedPreferences	n.java	SharedPreferences.Editor editorEdit = context.getSharedPreferences...
ELEVE	Token dans SharedPreferences	p.java	getSharedPreferences(String.format("com.google.firebase.auth
ELEVE	Token dans SharedPreferences	AddBeneficiary.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ApproveBeneficiary.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	BankLogin.java	Log.d("accesstoken", string)
ELEVE	Token dans SharedPreferences	BankLogin.java	SharedPreferences sharedPreferences = BankLogin.this.getShar...
ELEVE	Token dans SharedPreferences	BankLogin.java	sharedPreferences.edit().putString("accesstoken
ELEVE	Token dans SharedPreferences	Dashboard.java	SharedPreferences.Editor editorEdit = getApplicationContext().ge...
ELEVE	Token dans SharedPreferences	Myprofile.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ResetPassword.java	getSharedPreferences("jwt
ELEVE	Token dans SharedPreferences	ResetPassword.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	SendMoney.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ViewBalance.java	getSharedPreferences("jwt", 0).getString("accesstoken

3. Analyse Dynamique - Tests d'authentification

L'analyse dynamique evalue le comportement de l'application en cours d'exécution. Les tests portent sur la validité des tokens après déconnexion, la durée de vie, la rotation des tokens et la robustesse du mécanisme d'authentification.

Mode d'analyse :	API_DIRECT
Déconnexion effective :	Non teste
Durée de vie du token :	None min
Rotation des refresh tokens :	Non teste

Vulnérabilités détectées lors des tests :

INFO	DVBA non disponible sur http://localhost:3000. Démarrez le backend et relancez sans --skip-dyna
------	---

4. Scores MASVS par Domaine

Les scores reflètent le niveau de sécurité par domaine OWASP MASVS v2. Chaque vulnérabilité déduit des points selon sa sévérité. Un score inférieur à 4 indique un risque critique sur ce domaine.

MASVS-AUTH		0/10
MASVS-STORAGE		0/10
MASVS-CRYPTO		10/10
MASVS-NETWORK		9/10

5. Checklist de Sécurité (MASVS)

Checklist générée par IA (Claude) adaptée au type d'authentification détecté. Chaque contrôle est mappe à un chapitre OWASP MASVS pertinent. Ces points constituent le référentiel de vérification pour la revue de code.

- [] [MASVS-AUTH-1] Vérifier que les tokens JWT sont stockés dans le Keystore Android chiffré et non dans les SharedPreferences en clair
- [] [MASVS-AUTH-2] Vérifier que les tokens JWT ont une durée de vie limitée et qu'un mécanisme d'expiration côté serveur est effectivement appliqué
- [] [MASVS-AUTH-3] Vérifier que la rotation des tokens (refresh token) est implémentée et que les anciens tokens sont invalidés après rotation
- [] [MASVS-AUTH-4] Vérifier que la déconnexion entraîne une invalidation effective du token côté serveur (révocation) et non seulement une suppression locale
- [] [MASVS-AUTH-5] Vérifier que les tokens JWT ne sont pas rejouables après expiration ou déconnexion (protection anti-rejeu côté backend)
- [] [MASVS-STORAGE-1] Vérifier qu'aucun token d'accès (accesstoken JWT, token Firebase) n'est stocké dans les SharedPreferences sans chiffrement
- [] [MASVS-STORAGE-2] Vérifier qu'aucun secret ou mot de passe n'est codé en dur dans le code source de l'application (fichiers b.java et n1.java)
- [] [MASVS-STORAGE-3] Vérifier qu'aucune information sensible (tokens, credentials, erreurs d'authentification) n'est écrite dans les logs Android (Log.d, Log.e)
- [] [MASVS-STORAGE-4] Vérifier que le fichier de SharedPreferences contenant les tokens JWT est créé en mode MODE_PRIVATE et inaccessible aux autres applications
- [] [MASVS-NETWORK-1] Vérifier qu'aucune communication HTTP en clair n'est utilisée pour transmettre des tokens JWT ou des données d'authentification
- [] [MASVS-NETWORK-2] Vérifier que la configuration réseau (Network Security Config) interdit explicitement tout trafic en clair (cleartext) vers tous les domaines
- [] [MASVS-CRYPTO-1] Vérifier que le secret utilisé pour la signature des tokens JWT n'est pas codé en dur dans le code de l'application ou dans les ressources
- [] [MASVS-CRYPTO-2] Vérifier que l'algorithme de signature JWT utilisé est robuste (RS256 ou ES256) et que l'algorithme "none" est

explicitement rejeté

- [] [MASVS-CODE-4] Vérifier que les messages d'erreur liés à l'authentification biométrique (KeyguardManager, DeviceCredentialHandler) ne divulguent pas d'informations exploitables dans les logs de production

6. Criteres d'Acceptation de Securite

Les criteres d'acceptation definissent les conditions que le systeme doit satisfaire pour etre considere comme securise. Ils servent de base aux tests d'acceptation, aux revues de code et aux audits de conformite MASVS.

1. Le système DOIT stocker les tokens JWT exclusivement dans le Android Keystore ou dans un stockage chiffré équivalent afin d'empêcher leur extraction par une application tierce ou lors d'une compromission de l'appareil.
2. Le système NE DOIT PAS écrire de tokens d'accès, de secrets ou d'informations d'authentification dans les logs Android (Log.d, Log.e, Log.i) afin d'éviter leur exposition via des outils de lecture de logs (adb logcat).
3. Le système NE DOIT PAS contenir de secrets, de mots de passe ou de clés de signature JWT codés en dur dans le code source ou les ressources compilées afin de prévenir leur extraction par ingénierie inverse.
4. Le système DOIT invalider le token JWT côté serveur lors de chaque déconnexion explicite afin de garantir qu'un token intercepté ne puisse pas être réutilisé après la fin de session.
5. Le système DOIT implémenter une durée de vie maximale explicite pour les tokens JWT (access token ? 15 minutes, refresh token ? 24 heures) afin de limiter la fenêtre d'exploitation en cas de vol de token.
6. Le système DOIT implémenter un mécanisme de rotation des refresh tokens avec invalidation de l'ancien token à chaque renouvellement afin de détecter et bloquer les tentatives de rejeu.
7. Le système NE DOIT PAS transmettre de tokens JWT ou de credentials via des connexions HTTP non chiffrées afin de prévenir leur interception par une attaque de type man-in-the-middle.
8. Le système DOIT utiliser un algorithme de signature asymétrique (RS256 ou ES256) pour les tokens JWT et DOIT rejeter explicitement les tokens signés avec l'algorithme "none" afin de prévenir les attaques de falsification de tokens.